

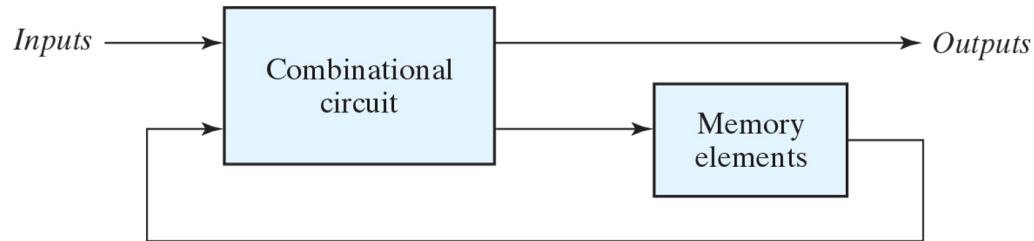


디지털공학

6 주차

5 순차 논리

❖ 개요

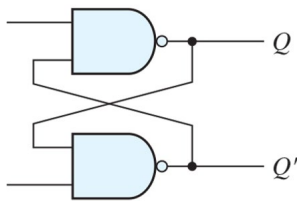


- 조합회로 + 저장회로
- 출력 값은 현재 입력 및 저장회로에 저장된 이전 값의 조합으로 결정
- 즉, 시간 순서에 따른 입력 값과 저장된 내부 상태 값에 따라 달라짐
- 동기식 / 비동기식 순차회로로 구분



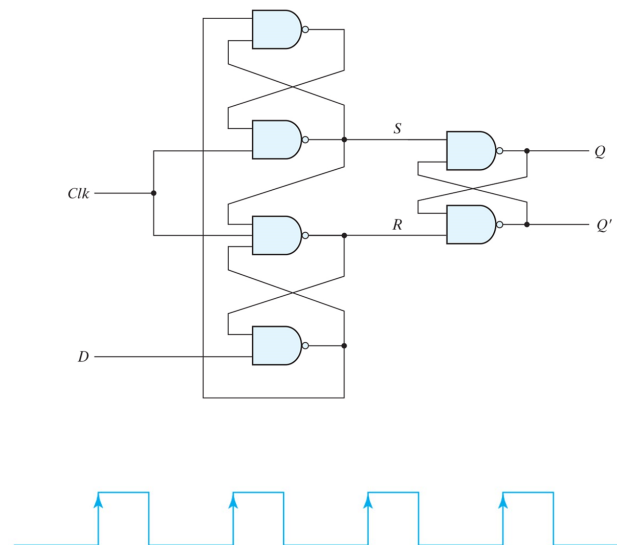
❖ 비동기식 순차논리

- 각 입력신호의 변화 순서에 의해서 출력신호가 변화됨
- 일반적으로 게이트형 비동기식 시스템의 경우 피드백이 있는 조합회로로 구성되어 회로가 불안정한 상태에 빠질 수 있음
- 실제로 많이 사용되지 않고 있어 본 학기에는 상세히 다루지 않을 예정임



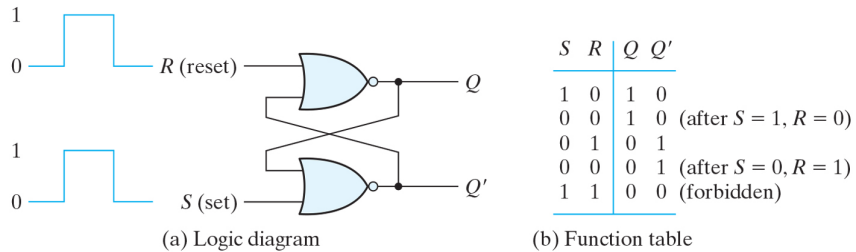
❖ 동기식 순차논리

- 클럭펄스(동기신호)에 의해 회로의 동작시점이 결정되며, 레벨응답/에지응답 형태로 구분

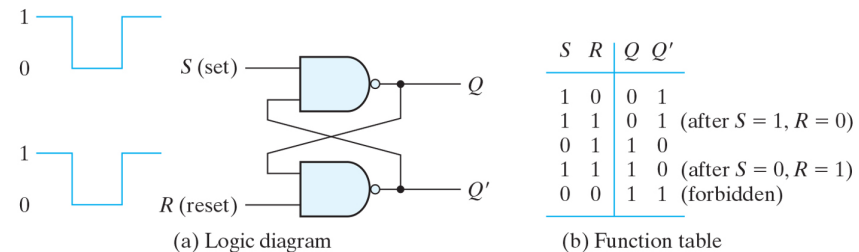


❖ 래치(Latch) – 비동기식 저장회로

- 저장요소는 상태변화 명령에 대한 입력이 있을 때 까지 상태를 유지할 수 있어야 함
- 래치는 입력 신호의 레벨에 의해 반응하는 비동기식 순차논리회로
- 일종의 쌍안정(bistable) 논리소자 또는 멀티바이브레이터(Multivibrator)라 칭함

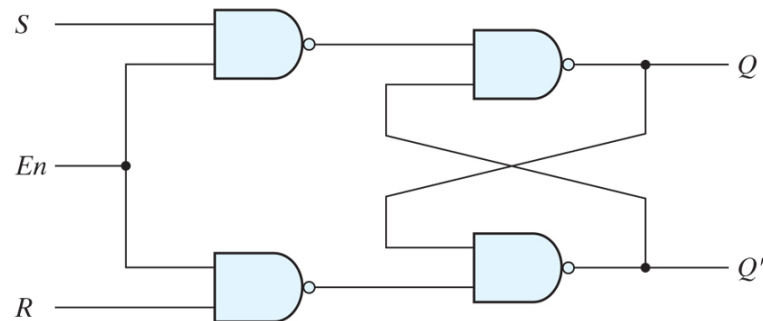
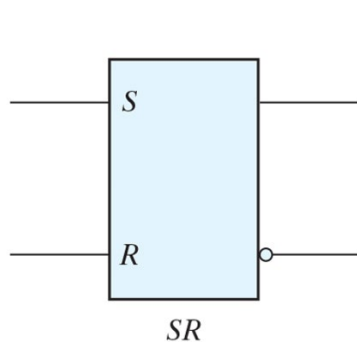


[Active High SR-Latch]



[Active Low SR-Latch]

[Enable 신호를 가지는 SR-Latch]

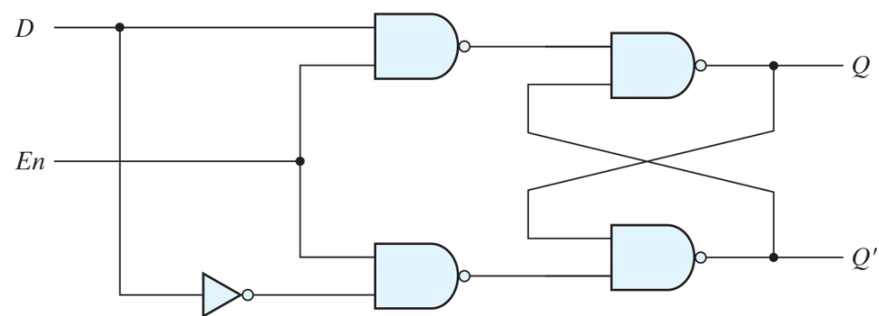
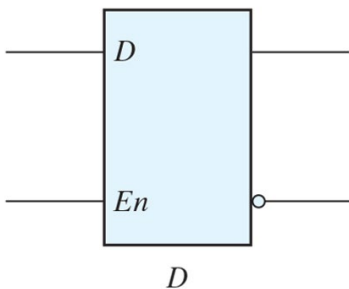


(a) Logic diagram

En	S	R	Next state of Q
0	X	X	No change
1	0	0	No change
1	0	1	$Q = 0$; reset state
1	1	0	$Q = 1$; set state
1	1	1	Indeterminate

(b) Function table

[D-Latch]



(a) Logic diagram

En	D	Next state of Q
0	X	No change
1	0	$Q = 0$; reset state
1	1	$Q = 1$; set state

(b) Function table

❖ 동기신호



(a) Response to positive level



(b) Positive-edge response

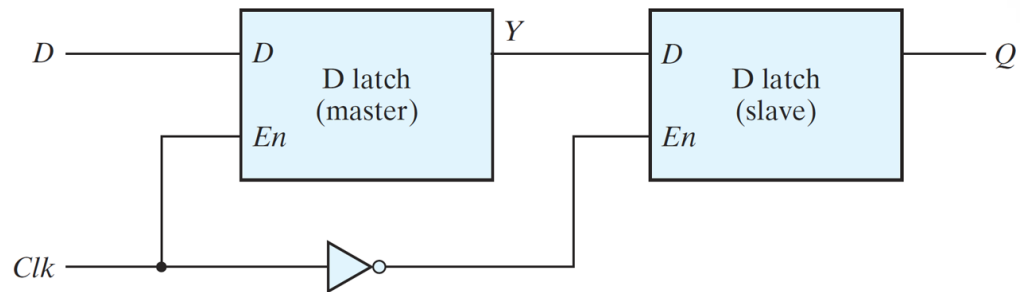


(c) Negative-edge response

- Clock신호의 레벨에 따라 동작여부가 결정되며, Active-High / Active-Low로 구분
- 상기 Level Response에서 Active Level을 유지하는 동안 입력신호에 변화가 생길 경우 계속해서 새로운 상태 변화가 나타나 출력이 불안정
- Level Response의 출력 불안정 상태 보완을 위해 입력신호 및 피드백 신호가 안정된 후 동기신호의 상승/하강 에지 시점에서만 출력신호 천이(Transition)가 발생하는 Edge Response 방식을 주로 사용
- Edge Response 방식을 사용할 경우 입력/내부/출력 신호와 클럭과의 타이밍을 고려하여 회로를 구성해야 하며, 입력이 일정한 값을 유지해야 하는 최소 시간을 Setup Time, Edge Trigger(Falling/Rising) 발생 후 입력을 유지해야 하는 최소 시간을 Hold Time이라 함

- 두 개의 D 래치와 한 개의 인버터를 이용한 D 플립플롭 구성

[Falling Edge D-FlipFlop]



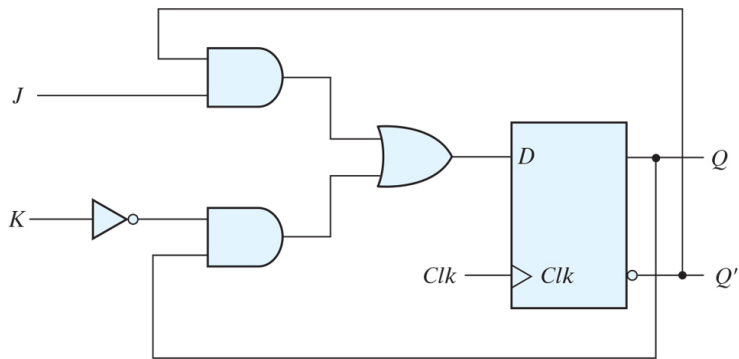
❖ 플립플롭(Flip-Flop) – 동기식 저장회로

- 디지털 시스템 설계에서 가장 많이 사용되는 플립플롭은 JK 플립플롭과 T 플립플롭이며, D 플립플롭과 외부 논리회로를 이용하여 만들어짐
- D 플립플롭이 가장 적은 수의 게이트를 이용해 구현할 수 있어 가장 효율적
- 플립플롭은 Set / Reset / Complement 동작을 수행할 수 있으며,
 - JK 플립플롭은 3가지 기능 모두 수행
 - T 플립플롭은 Complement 동작만 수행

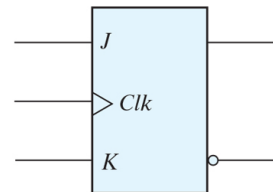
❖ JK 플립플롭

JK Flip-Flop			
J	K	Q(t + 1)	
0	0	Q(t)	No change
0	1	0	Reset
1	0	1	Set
1	1	Q'(t)	Complement

$$D = JQ' + K'Q$$



(a) Circuit diagram



(b) Graphic symbol

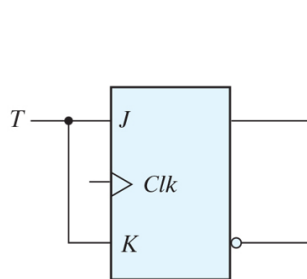


❖ T 플립플롭

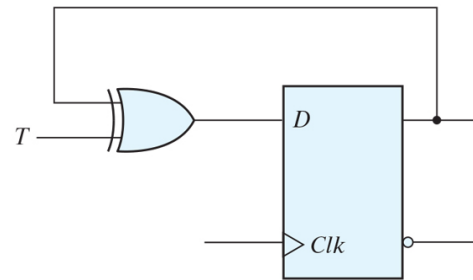
T Flip-Flop

T	Q(t + 1)	
0	$Q(t)$	No change
1	$Q'(t)$	Complement

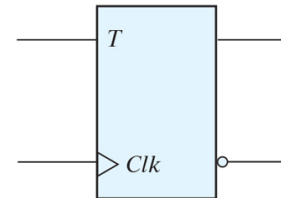
$$D = T \oplus Q$$



(a) From JK flip-flop



(b) From D flip-flop

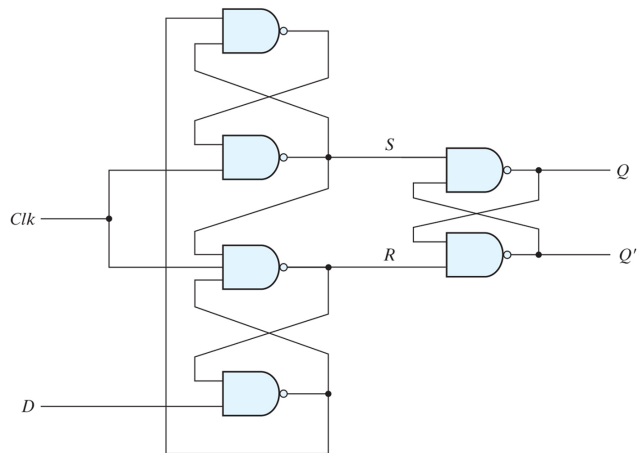


(c) Graphic symbol

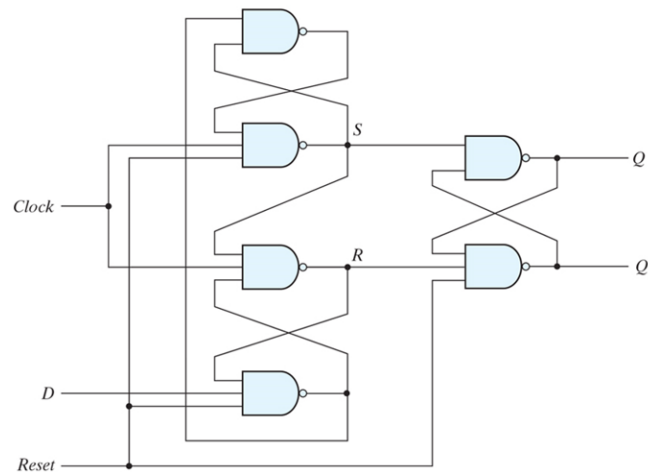


❖ 직접입력

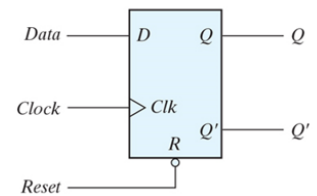
- 일부 플립플롭의 경우 필요에 따라 특정 상태로 설정할 수 있는 비동기 입력을 이용 (예. 디지털 시스템에 전원이 인가된 경우 초기상태를 알 수 없음)
- 플립플롭의 상태를 1로 만드는 경우를 Preset, 0으로 만드는 상태를 Clear 라 칭함



[Rising Edge D-FlipFlop]



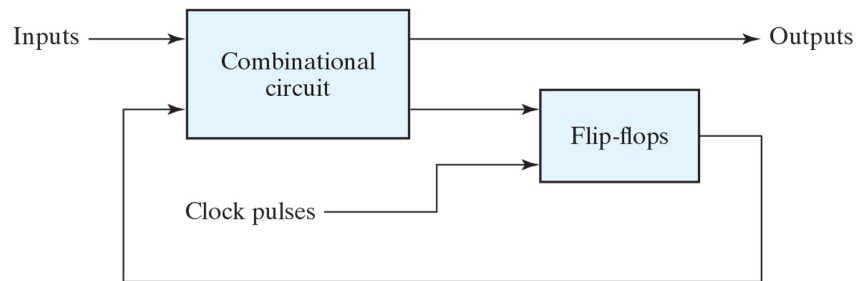
[Rising Edge D-FlipFlop with asynchronous Reset]



R	Clk	D	Q	Q'
0	X	X	0	1
1	↑	0	0	1
1	↑	1	1	0

❖ 클럭형 순차회로 해석

- 클럭에 의해 동작하는 동기식 순차회로의 동작은 입력 / 출력 / 플립플롭의 상태에 의해 결정
- 플립플롭이 클럭에 의해 동작하므로 플립플롭의 현재상태가 다음 클럭의 회로 동작에 영향을 줌
- 이로 인해 순차 회로의 해석은 입력 / 출력 / 내부상태를 시간 순서로 해석하여 상태표, 상태천이도로 표현
- 회로도 -> 관계식 -> 상태표 -> 상태천이도

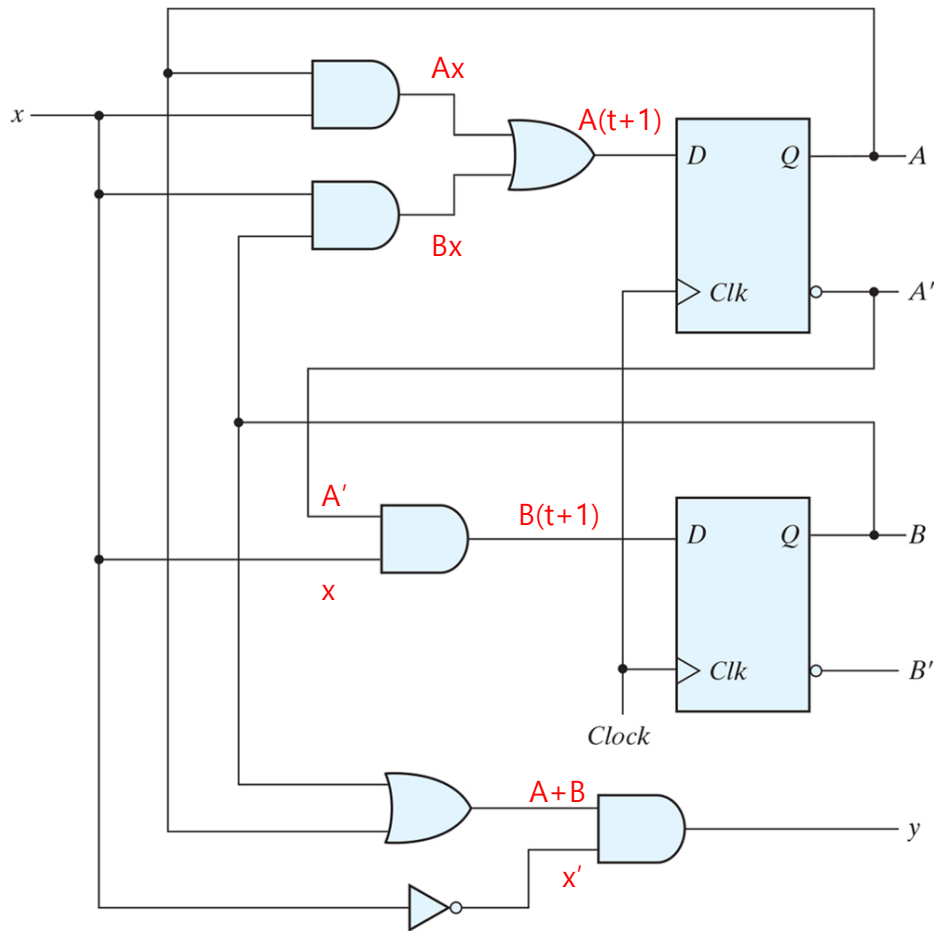


(a) Block diagram



(b) Timing diagram of clock pulses

[클럭형 순차회로 해석 예제]



$$A(t+1) = A(t)x(t) + B(t)x(t)$$

$$B(t+1) = A'(t)x(t)$$

$$y(t) = [A(t) + B(t)] x'(t)$$

$$A(t+1) = Ax + Bx$$

$$B(t+1) = A'x$$

$$y = (A+B)x'$$

$$A(t+1) = Ax + Bx$$

$$B(t+1) = A'x$$

$$y = (A+B)x'$$

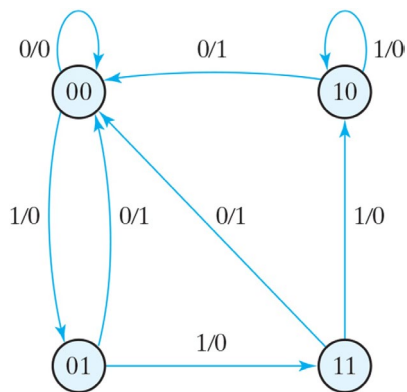
Present State		Input	Next State		Output
<i>A</i>	<i>B</i>		<i>A</i>	<i>B</i>	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0



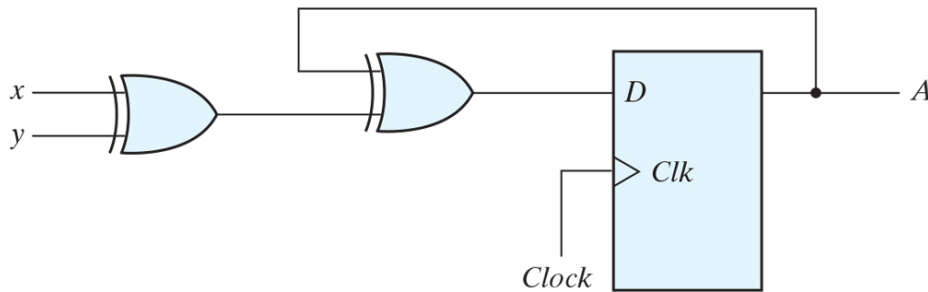
Present State		Input	Next State		Output
A	B		A	B	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

현재입력/현재출력

플립플롭
상태



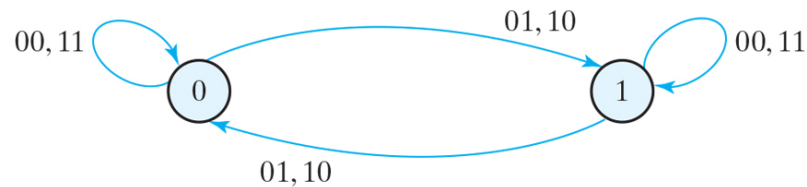
[클럭형 순차회로 해석 예제 - 2]



(a) Circuit diagram

Present state	Inputs		Next state
A	x	y	A
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(b) State table

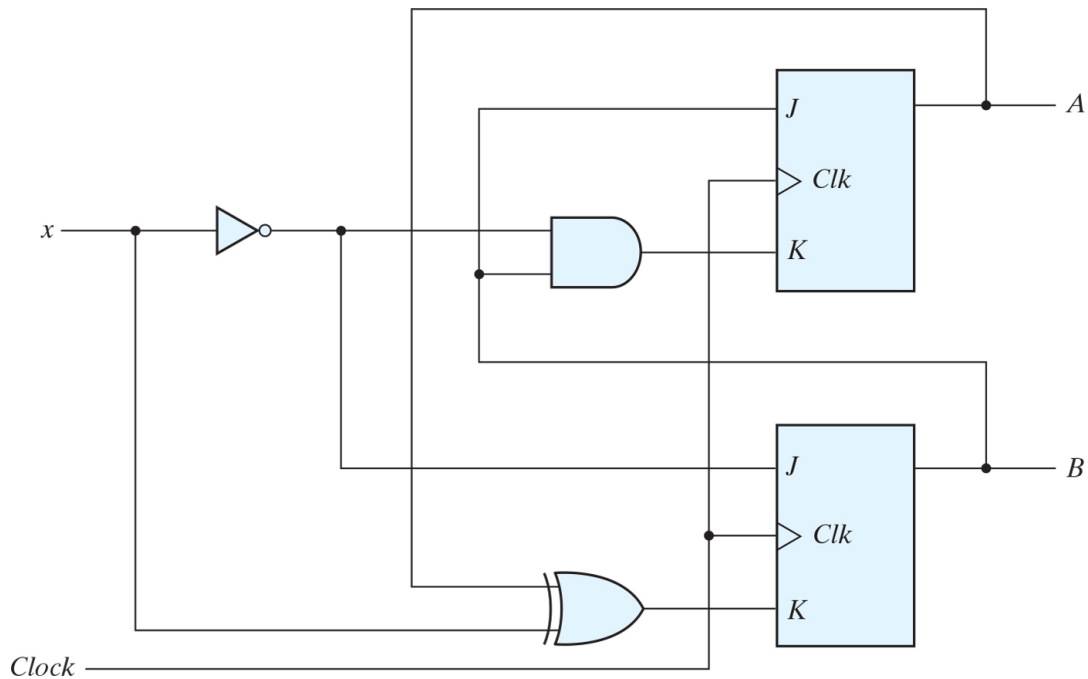


(c) State diagram

현재입력(xy), 현재입력(xy)

플립플롭
상태

[클럭형 순차회로 해석 예제 - JK 플립플롭]

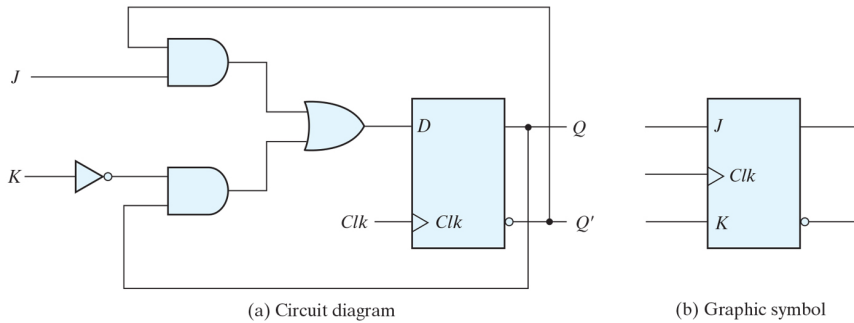


$$J_A = B$$

$$K_A = Bx'$$

$$J_B = x'$$

$$K_B = A'x + Ax'$$



$$D = JQ' + K'Q$$

$$\begin{aligned} J_A &= B \\ K_A &= Bx' \end{aligned}$$

$$A(t+1) = JA' + K'A = BA' + (Bx')'A = A'B + (B' + x)A = A'B + AB' + Ax$$

$$\begin{aligned} J_B &= x' \\ K_B &= A'x + Ax' \end{aligned}$$

$$\begin{aligned} B(t+1) &= JB' + K'B = x'B' + (A'x + Ax')'B = x'B' + (Ax + A'x')B \\ &= x'B' + Abx + A'Bx' \end{aligned}$$

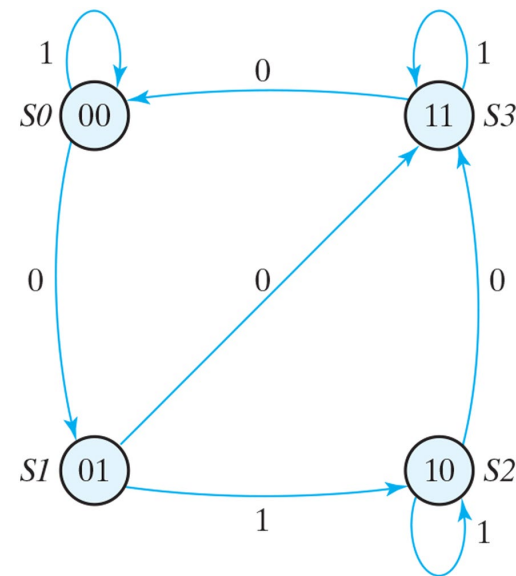
$$J_A = B \quad J_B = x'$$

$$K_A = Bx' \quad K_B = A'x + Ax'$$

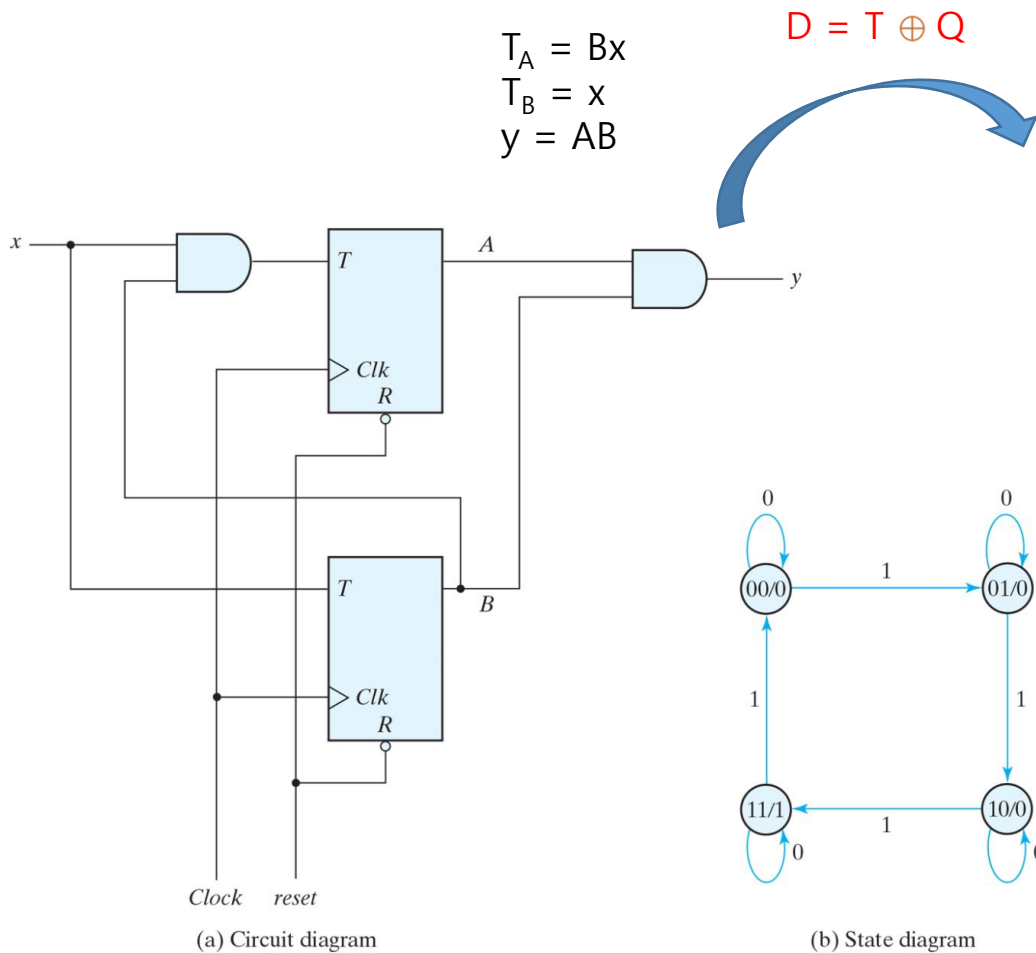
Present State		Input	Next State		Flip-Flop Inputs			
A	B		A	B	J_A	K_A	J_B	K_B
0	0	0	0	1	0	0	1	0
0	0	1	0	0	0	0	0	1
0	1	0	1	1	1	1	1	0
0	1	1	1	0	1	0	0	1
1	0	0	1	1	0	0	1	1
1	0	1	1	0	0	0	0	0
1	1	0	0	0	1	1	1	1
1	1	1	1	1	1	0	0	0

$$A(t+1) = A'B + AB' + Ax$$

$$B(t+1) = x'B' + Abx + A'Bx'$$



[클럭형 순차회로 해석 예제 - T 플립플롭]



$$A(t+1) = (Bx)'A + (Bx)A'$$

$$= AB' + Ax' + A'Bx$$

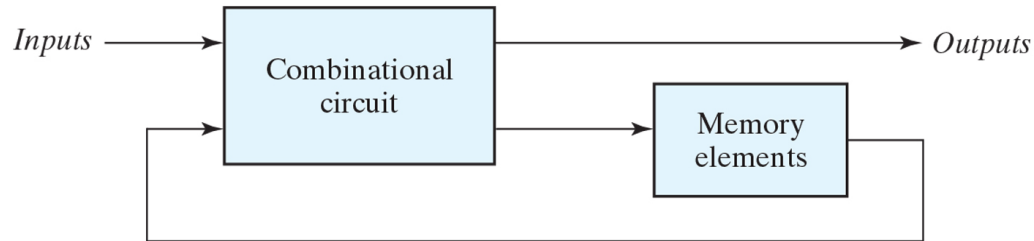
$$B(t+1) = x'B + xB'$$

Present State		Input x	Next State		Output y
A	B		A	B	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	1

현재입력(x)

플립플롭 상태 / 출력(y)

❖ 개요

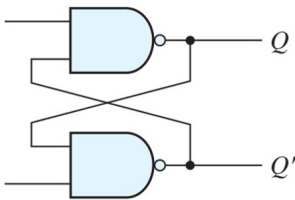


- 조합회로 + 저장회로
- 출력 값은 현재 입력 및 저장회로에 저장된 이전 값의 조합으로 결정
- 즉, 시간 순서에 따른 입력 값과 저장된 내부 상태 값에 따라 달라짐
- 동기식 / 비동기식 순차회로로 구분



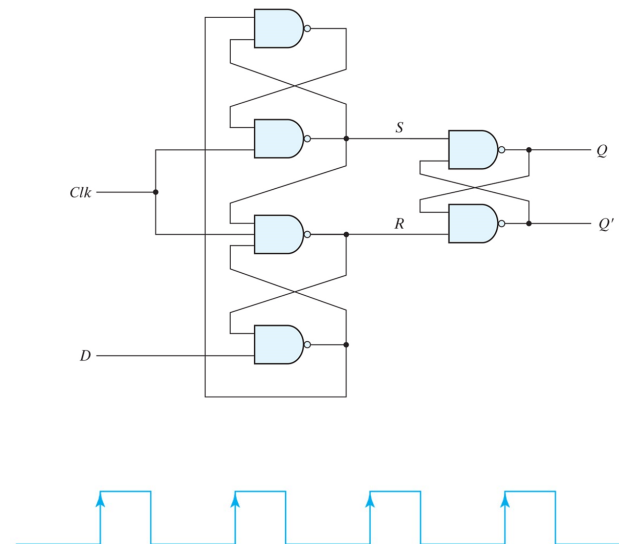
❖ 비동기식 순차논리

- 각 입력신호의 변화 순서에 의해서 출력신호가 변화됨
- 일반적으로 게이트형 비동기식 시스템의 경우 피드백이 있는 조합회로로 구성되어 회로가 불안정한 상태에 빠질 수 있음
- 실제로 많이 사용되지 않고 있어 본 학기에는 상세히 다루지 않을 예정임



❖ 동기식 순차논리

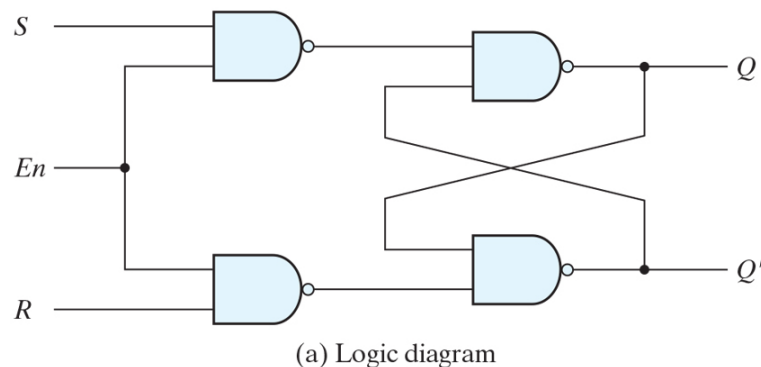
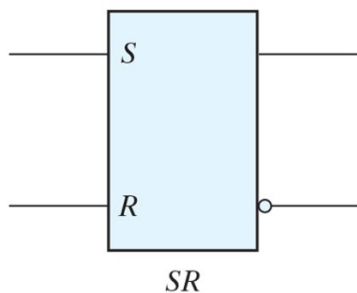
- 클럭펄스(동기신호)에 의해 회로의 동작시점이 결정되며, 레벨응답/에지응답 형태로 구분



❖ 래치(Latch) – 비동기식 저장회로

- 저장요소는 상태변화 명령에 대한 입력이 있을 때 까지 상태를 유지할 수 있어야 함
- 래치는 입력 신호의 레벨에 의해 반응하는 비동기식 순차논리회로

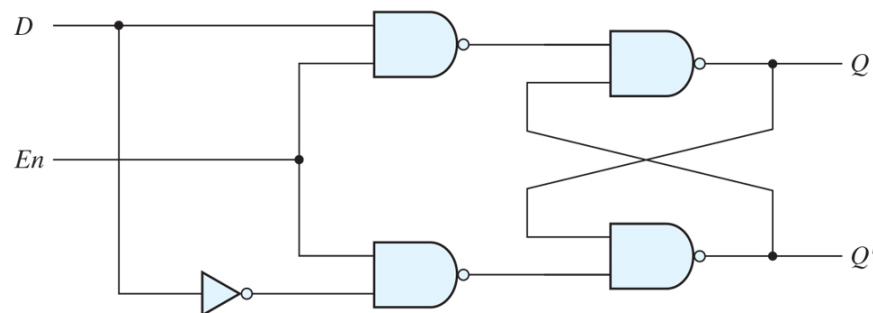
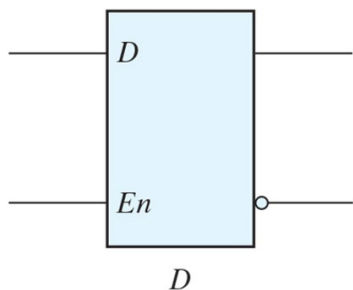
[Enable 신호를 가지는 SR-Latch]



En	S	R	Next state of Q
0	X	X	No change
1	0	0	No change
1	0	1	$Q = 0$; reset state
1	1	0	$Q = 1$; set state
1	1	1	Indeterminate

(b) Function table

[D-Latch]

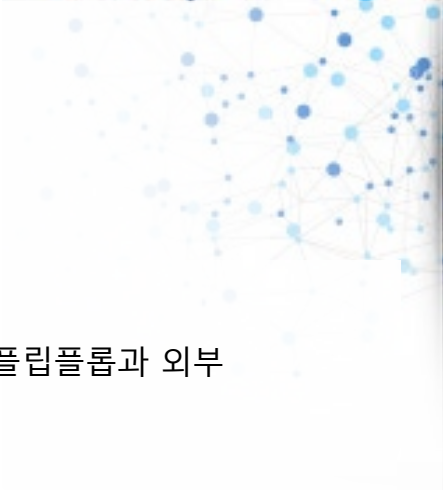


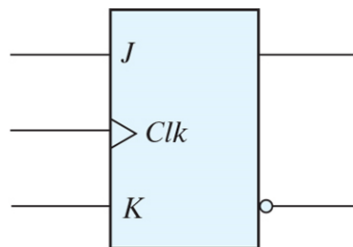
En	D	Next state of Q
0	X	No change
1	0	$Q = 0$; reset state
1	1	$Q = 1$; set state

(b) Function table

❖ 플립플롭(Flip-Flop) – 동기식 저장회로

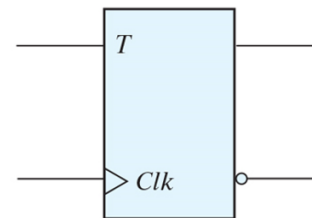
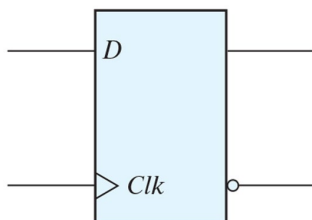
- 디지털 시스템 설계에서 가장 많이 사용되는 플립플롭은 JK 플립플롭과 T 플립플롭이며, D 플립플롭과 외부 논리회로를 이용하여 만들어짐
- D 플립플롭이 가장 적은 수의 게이트를 이용해 구현할 수 있어 가장 효율적
- 플립플롭은 Set / Reset / Complement 동작을 수행할 수 있으며,
 - JK 플립플롭은 3가지 기능 모두 수행
 - T 플립플롭은 Complement 동작만 수행





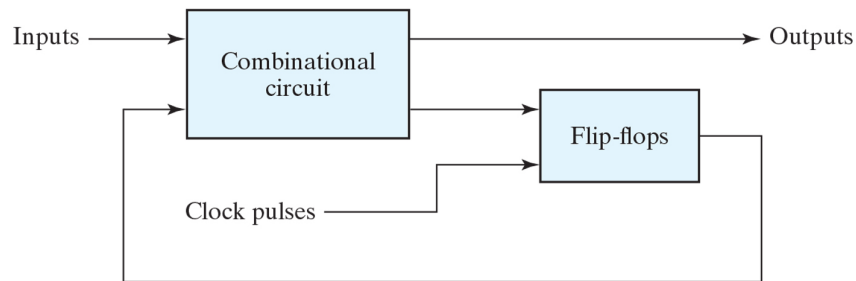
<i>J/K</i> Flip-Flop			
<i>J</i>	<i>K</i>	$Q(t + 1)$	
0	0	$Q(t)$	No change
0	1	0	Reset
1	0	1	Set
1	1	$Q'(t)$	Complement

<i>D</i> Flip-Flop			<i>T</i> Flip-Flop		
<i>D</i>	$Q(t + 1)$		<i>T</i>	$Q(t + 1)$	
0	0	Reset	0	$Q(t)$	No change
1	1	Set	1	$Q'(t)$	Complement



❖ 클럭형 순차회로 해석

- 클럭에 의해 동작하는 동기식 순차회로의 동작은 입력 / 출력 / 플립플롭의 상태에 의해 결정
- 플립플롭이 클럭에 의해 동작하므로 플립플롭의 현재상태가 다음 클럭의 회로 동작에 영향을 줌
- 이로 인해 순차 회로의 해석은 입력 / 출력 / 내부상태를 시간 순서로 해석하여 상태표, 상태천이도로 표현
- 회로도 -> 관계식 -> 상태표 -> 상태천이도



(a) Block diagram

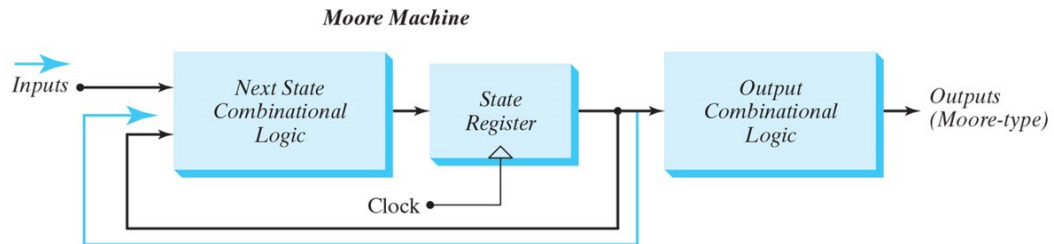


(b) Timing diagram of clock pulses

❖ 유한상태머신(FSM, Finite State Machine)의 표현 방법

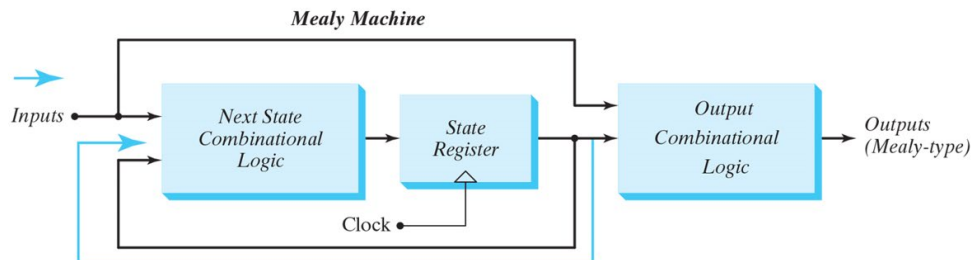
• Moore Machine

- ✓ 출력 값이 현재 상태 값에 의해서만 결정되는 모델
- ✓ 상태를 나타내는 플립플롭의 출력에만 의존하므로 클럭과 동기화 됨

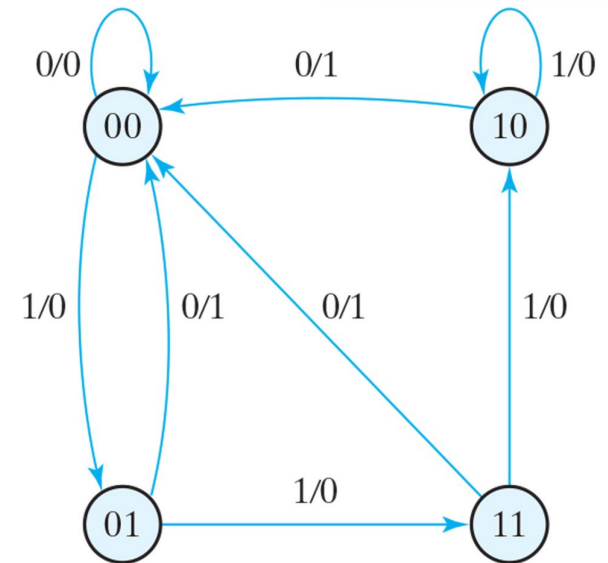
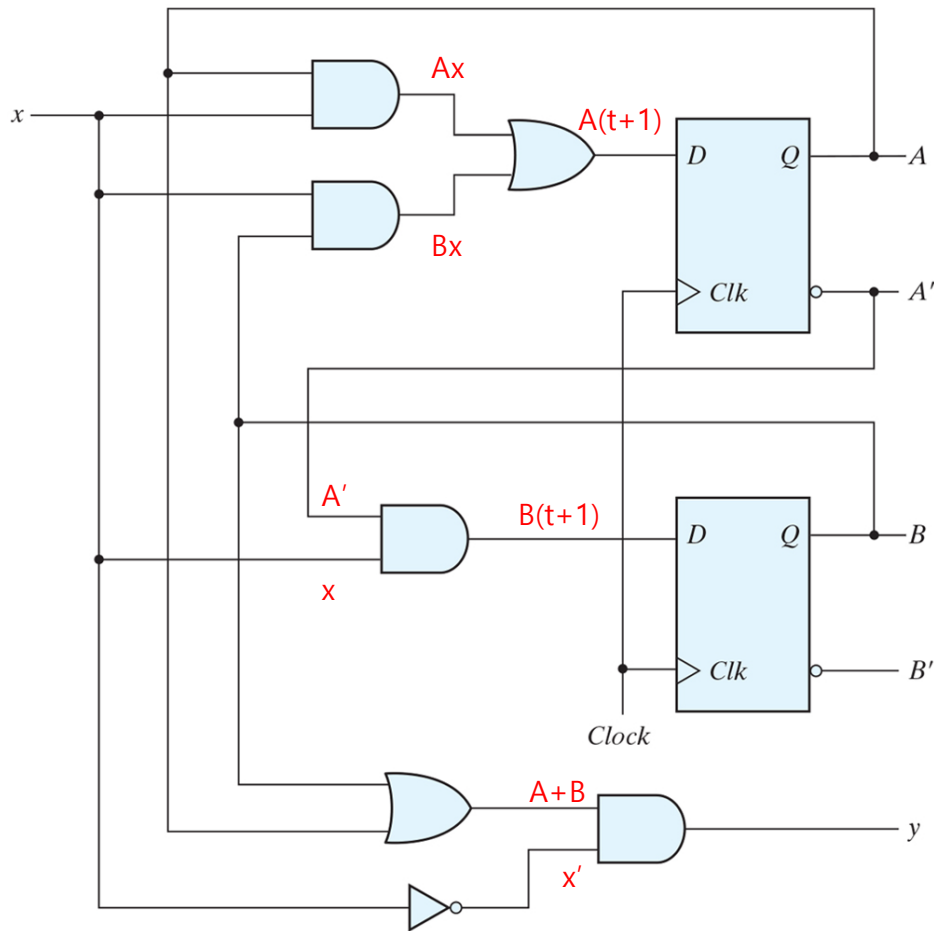


• Mealy Machine

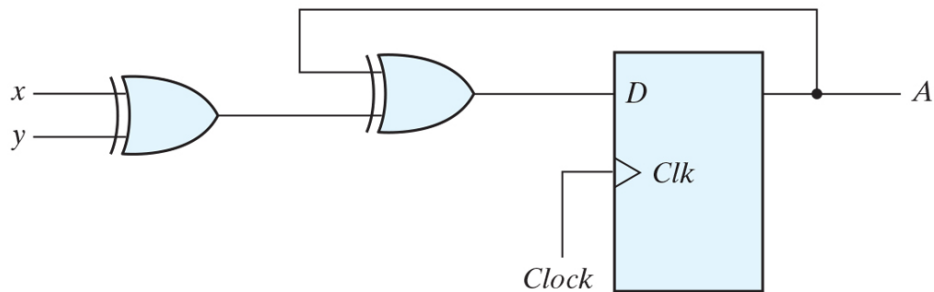
- ✓ 출력 값이 현재상태 + 입력신호의 조합에 의해 결정되는 모델
- ✓ 비동기 입력신호가 사용될 경우 입력신호의 천이와 클럭 사이클 간의 시간 지연으로 글리치(glitch, 하드웨어에 의한 시스템의 일시적 오류)가 발생할 수 있음
- ✓ 동기화 시스템 설계 시 글리치 문제 해결을 위해 시스템 클럭의 활성화 에지 직전의 비동기 입력값을 샘플링하여 사용



[클럭형 순차회로 해석 예제]



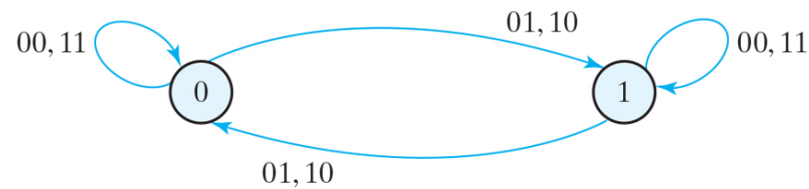
[클럭형 순차회로 해석 예제 - 2]



(a) Circuit diagram

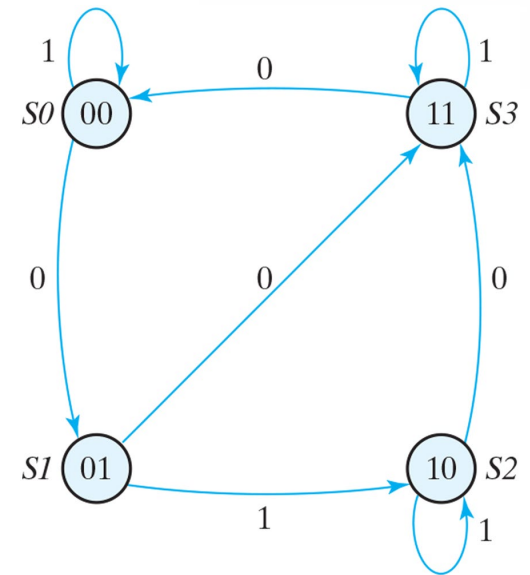
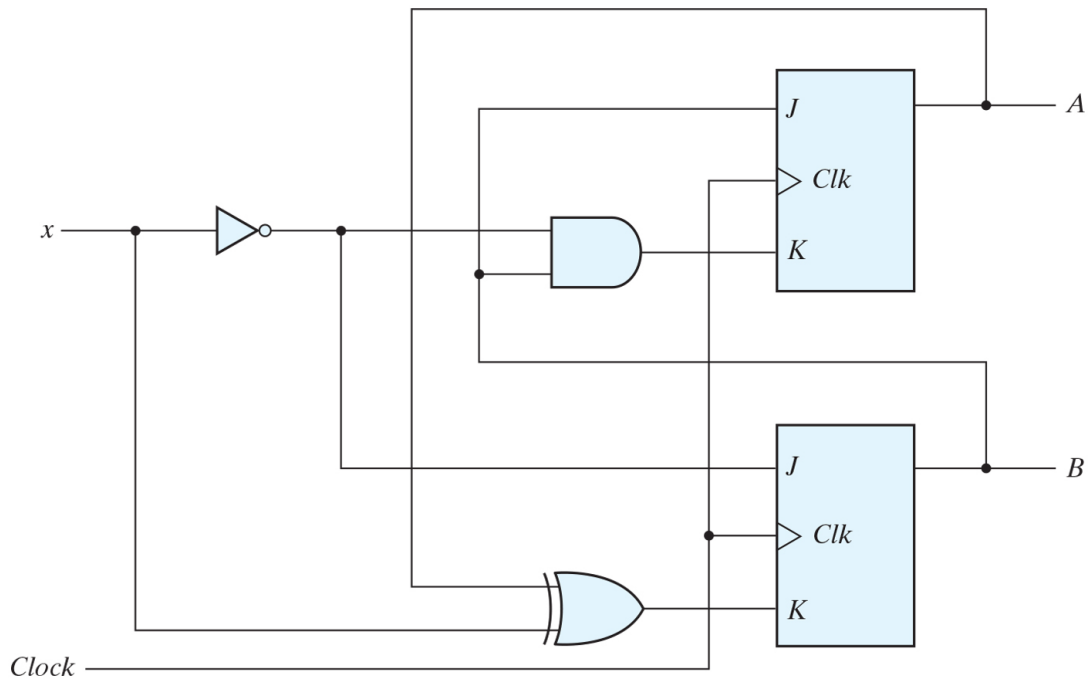
Present state	Inputs		Next state
A	x	y	A
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(b) State table

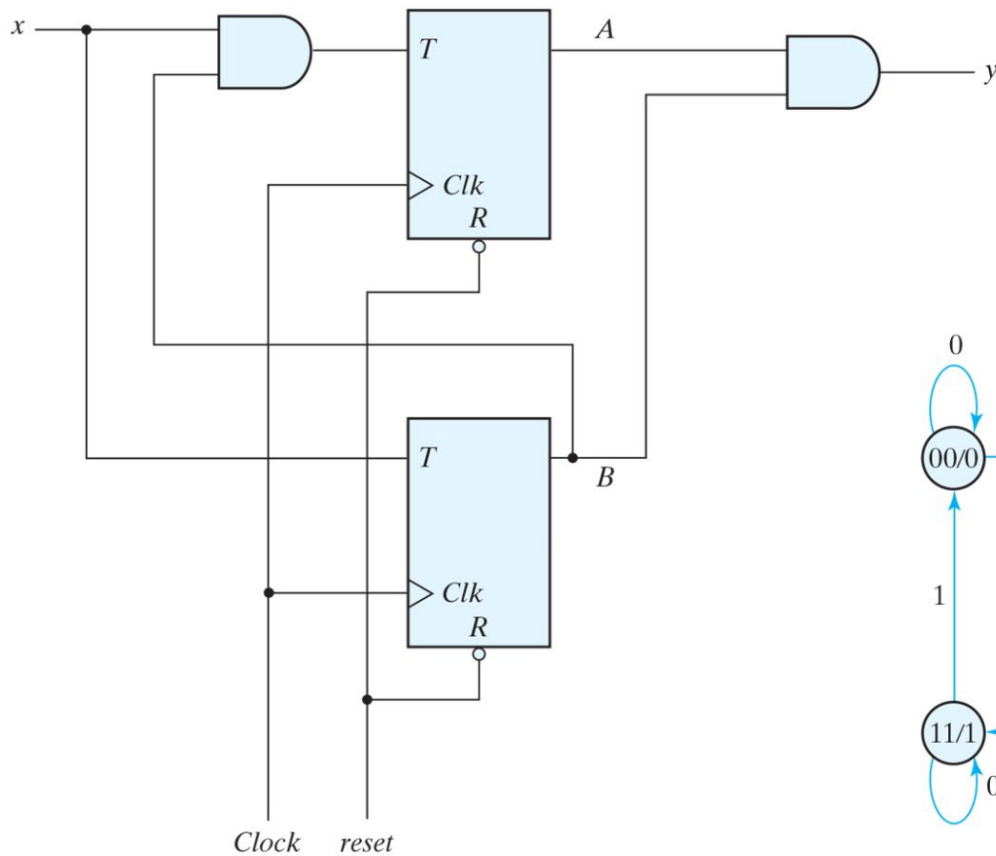


(c) State diagram

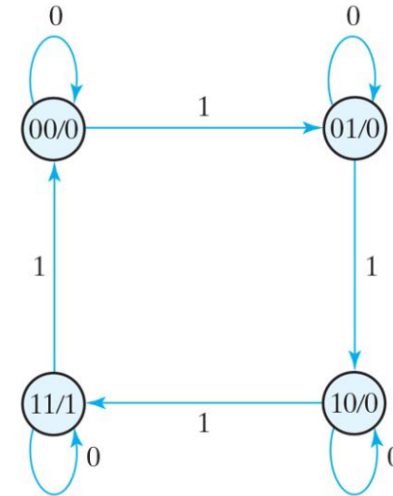
[클럭형 순차회로 해석 예제 - JK 플립플롭]



[클럭형 순차회로 해석 예제 - T 플립플롭]



(a) Circuit diagram



(b) State diagram

❖ 설계

1. 시스템 사양 및 기능 분석을 통해 상태도 도출
2. 상태축소 가능 시 축소
3. 상태에 대한 2진 코드 할당
4. 2진 코드 상태표 작성
5. 플립플롭 종류 선택
6. 선택한 플립플롭에 따른 논리식 도출
7. 논리회로 작성

정형화 된 단계로서 자동화 가능하며
실제 설계에서는 소프트웨어 툴 사용



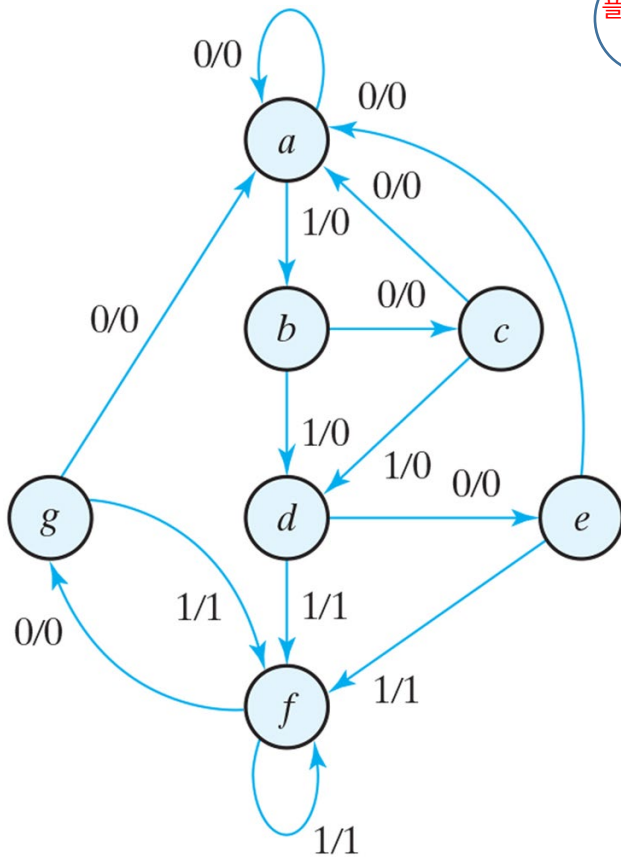
❖ 상태축소

1. 시스템 사양 및 기능 분석을 통해 상태도 도출
2. 상태축소 가능 시 축소
3. 상태에 대한 2진 코드 할당
4. 2진 코드 상태표 작성
5. 플립플롭 종류 선택
6. 선택한 플립플롭에 따른 논리식 도출
7. 논리회로 작성



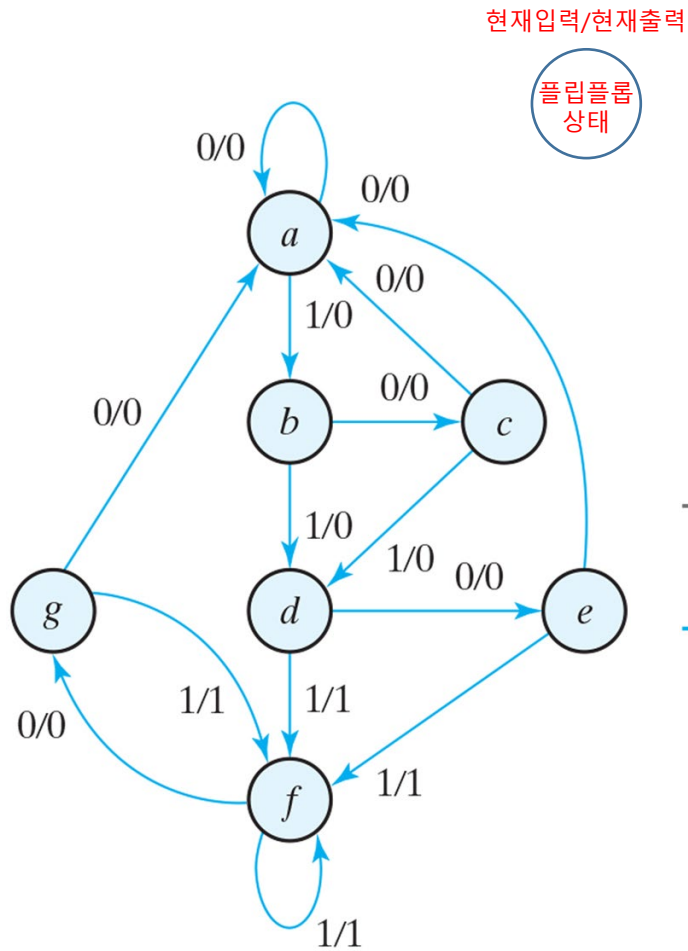
❖ 상태 축소

현재입력/현재출력

플립플롭
상태

상태	a										
입력	0	1	0	1	0	1	1	0	1	0	0
출력											

❖ 상태 축소



Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	g	f	0	1
g	a	f	0	1



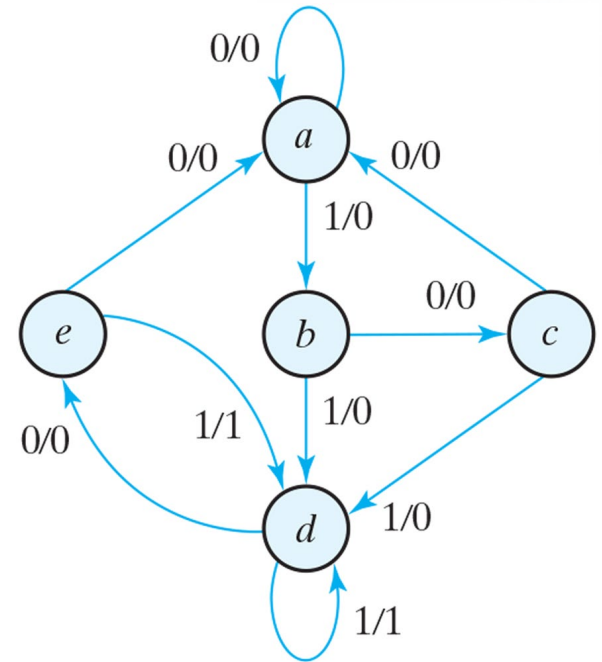
Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	g	f	0	1
g	a	f	0	1

$g = e$

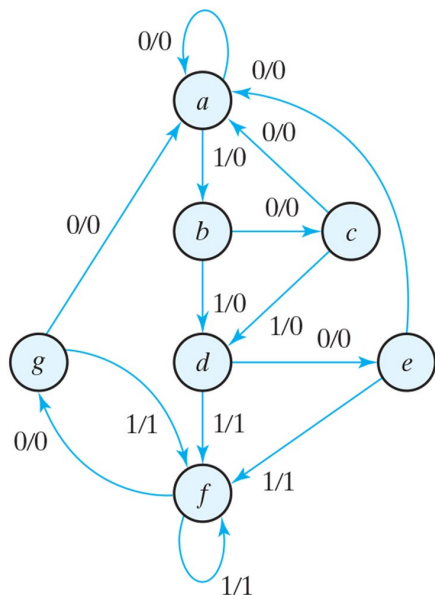
Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	e	f	0	1

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	e	f	0	1

$f = d$



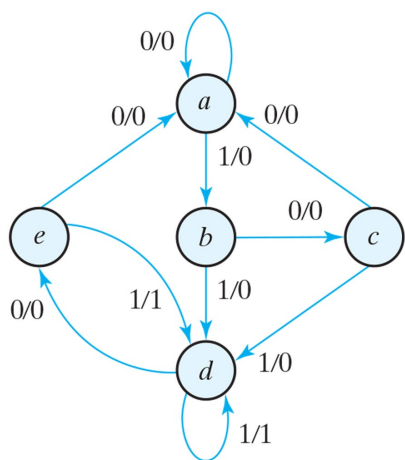
Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	d	0	1
e	a	d	0	1



상태	a	a	b	c	d	e	f	f	g	f	g	a
입력	0	1	0	1	0	1	1	0	1	0	0	
출력	0	0	0	0	0	1	1	0	1	0	0	

g = e

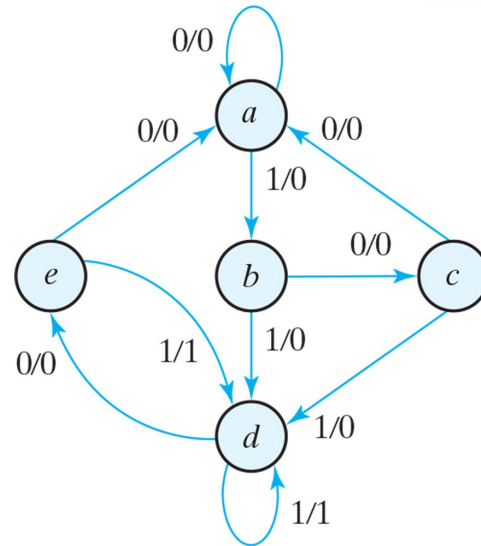
f = d



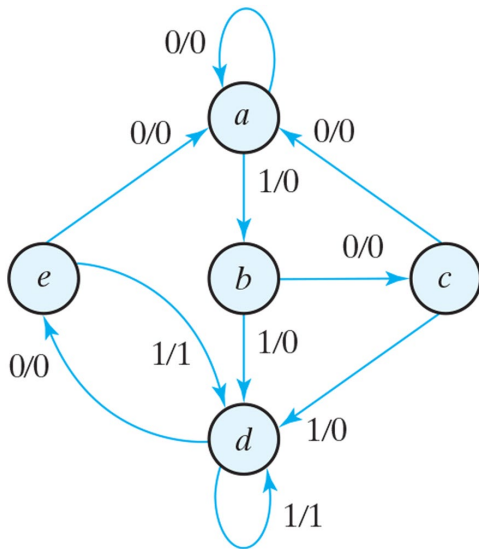
상태	a
입력	0 1 0 1 0 1 1 0 1 0 0
출력	

❖ 상태 할당

1. 시스템 사양 및 기능 분석을 통해 상태도 도출
2. 상태축소 가능 시 축소
3. 상태에 대한 2진 코드 할당
4. 2진 코드 상태표 작성
5. 플립플롭 종류 선택
6. 선택한 플립플롭에 따른 논리식 도출
7. 논리회로 작성



State	Assignment 1, Binary	Assignment 2, Gray Code	Assignment 3, One-Hot
<i>a</i>	000	000	00001
<i>b</i>	001	001	00010
<i>c</i>	010	011	00100
<i>d</i>	011	010	01000
<i>e</i>	100	110	10000



Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
<i>a</i>	<i>a</i>	<i>b</i>	0	0
<i>b</i>	<i>c</i>	<i>d</i>	0	0
<i>c</i>	<i>a</i>	<i>d</i>	0	0
<i>d</i>	<i>e</i>	<i>d</i>	0	1
<i>e</i>	<i>a</i>	<i>d</i>	0	1

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
000	000	001	0	0
001	010	011	0	0
010	000	011	0	0
011	100	011	0	1
100	000	011	0	1

JK Flip-Flop			
J	K	Q(t + 1)	
0	0	Q(t)	No change
0	1	0	Reset
1	0	1	Set
1	1	Q'(t)	Complement

D Flip-Flop			T Flip-Flop		
D	Q(t + 1)		T	Q(t + 1)	
0	0	Reset	0	Q(t)	No change
1	1	Set	1	Q'(t)	Complement

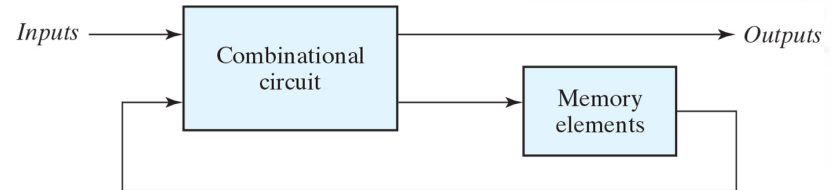


Q(t)	Q(t = 1)	J	K	Q(t)	Q(t = 1)	T
0	0	0	X	0	0	0
0	1	1	X	0	1	1
1	0	X	1	1	0	1
1	1	X	0	1	1	0

(a) JK Flip-Flop

(b) T Flip-Flop

1. 시스템 사양 및 기능 분석을 통해 상태도 도출
2. 상태축소 가능 시 축소
3. 상태에 대한 2진 코드 할당
4. 2진 코드 상태표 작성
5. 플립플롭 종류 선택
6. 선택한 플립플롭에 따른 논리식 도출
7. 논리회로 작성



Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
000	000	001	0	0
001	010	011	0	0
010	000	011	0	0
011	100	011	0	1
100	000	011	0	1

각각의 논리식 도출을 위해 카노맵 사용
후 논리회로 작성

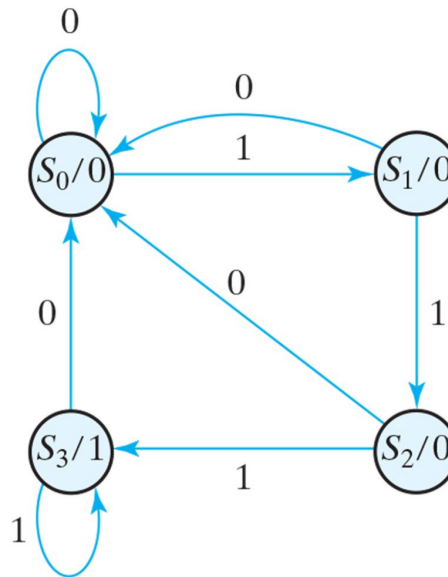
❖ 설계 예제 - D 플립플롭

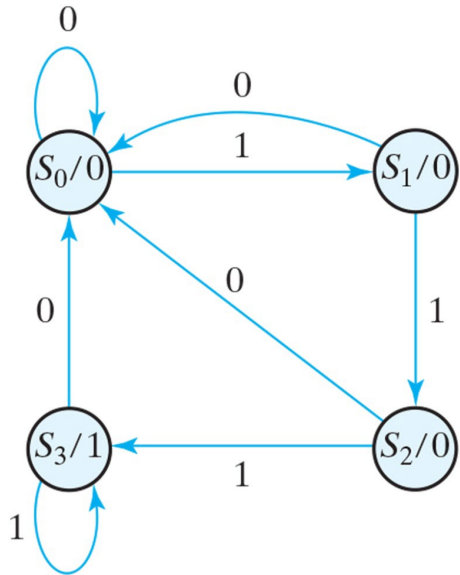
입력 : Bit Stream (1-bit)

출력 : y (1-bit), 상태(2-bit)

기능 : 입력에 1의 값이 연속으로 3개 이상 입력될 경우 출력을 Set

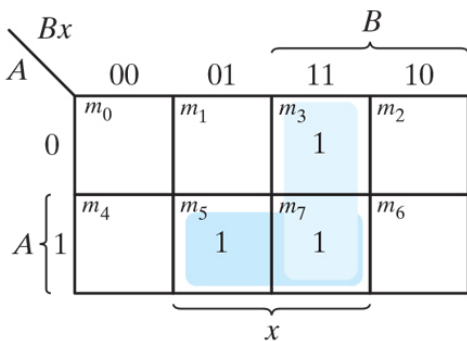
플립플롭 : D 플립플롭 사용



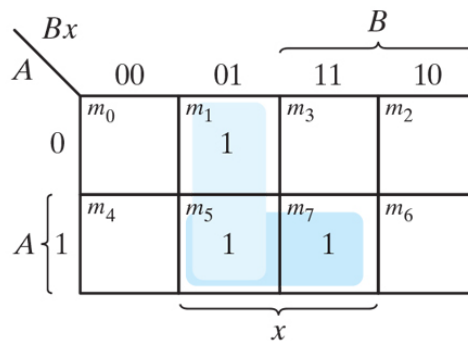


Present State		Input x	Next State		Output y
A	B		A	B	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	1	1	1	1

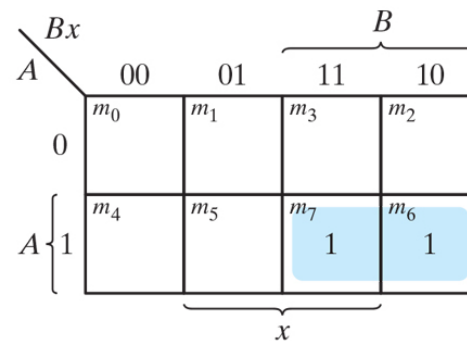
Present State		Input	Next State		Output
A	B		A	B	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	1	1	1	1



$$D_A = Ax + Bx$$



$$D_B = Ax + B'x$$

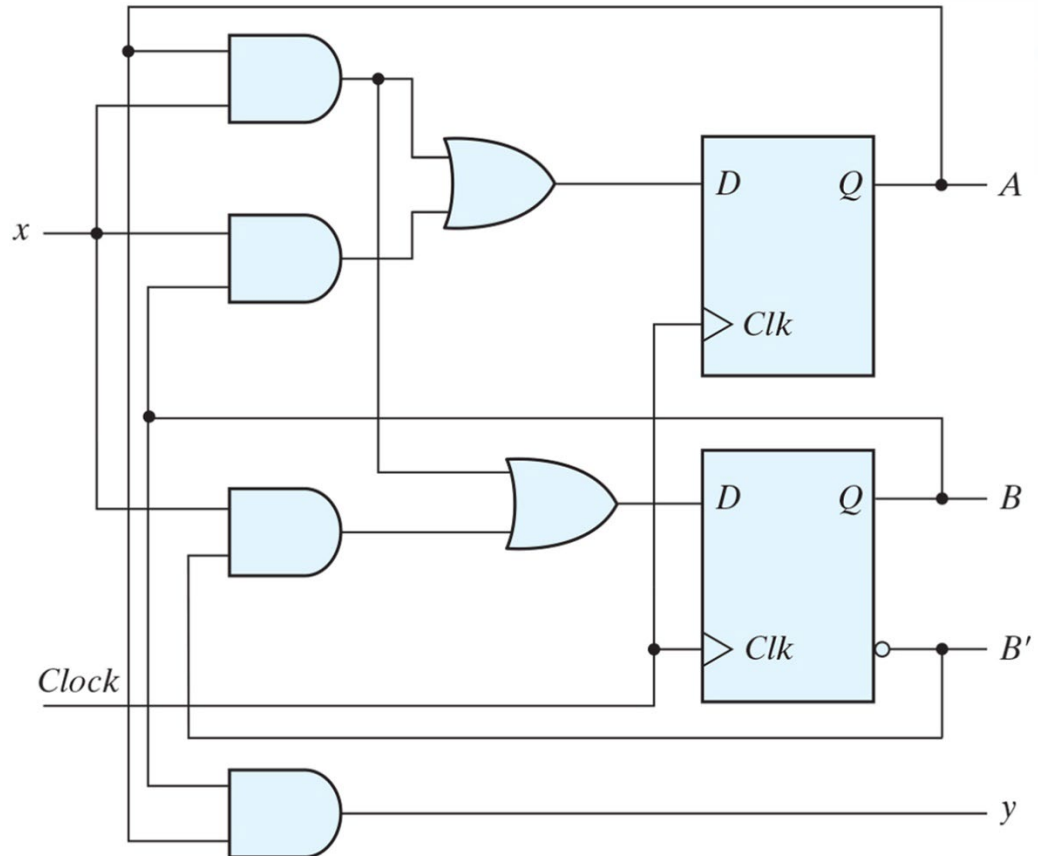


$$y = AB$$

$$D_A = Ax + Bx$$

$$D_B = Ax + B'x$$

$$y = AB$$

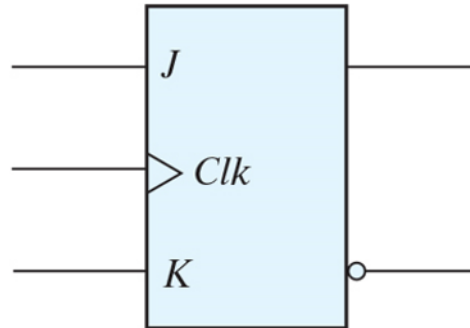


❖ 설계 예제 – JK 플립플롭

1. 시스템 사양 및 기능 분석을 통해 상태도 도출
2. 상태축소 가능 시 축소
3. 상태에 대한 2진 코드 할당
4. 2진 코드 상태표 작성
5. 플립플롭 종류 선택
6. 선택한 플립플롭에 따른 논리식 도출
7. 논리회로 작성

JK플립플롭 및 T플립플롭 역시 1~4 과정은 동일하게 거침

Present State		Input	Next State	
A	B		A	B
0	0	0	0	0
0	0	1	0	1
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	1	1
1	1	0	1	1
1	1	1	0	0



Present State		Input x	Next State		Flip-Flop Inputs			
A	B		A	B	J_A	K_A	J_B	K_B
0	0	0	0	0	0	X	0	X
0	0	1	0	1	0	X	1	X
0	1	0	1	0	1	X	X	1
0	1	1	0	1	0	X	X	0
1	0	0	1	0	X	0	0	X
1	0	1	1	1	X	0	1	X
1	1	0	1	1	X	0	X	0
1	1	1	0	0	X	1	X	1

$Q(t)$	$Q(t = 1)$	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Present State		Input	Next State		Flip-Flop Inputs			
A	B		A	B	J_A	K_A	J_B	K_B
0	0	0	0	0	0	X	0	X
0	0	1	0	1	0	X	1	X
0	1	0	1	0	1	X	X	1
0	1	1	0	1	0	X	X	0
1	0	0	1	0	X	0	0	X
1	0	1	1	1	X	0	1	X
1	1	0	1	1	X	0	X	0
1	1	1	0	0	X	1	X	1

		B			
		00	01	11	10
A	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6
					1
		X	X	X	X

$J_A = Bx'$

		B			
		00	01	11	10
A	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6
		X	X	X	X
				1	

$K_A = Bx$

		B			
		00	01	11	10
A	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6
			1	X	X
			1	X	X

$J_B = x$

		B			
		00	01	11	10
A	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6
		X	X		1
		X	X	1	

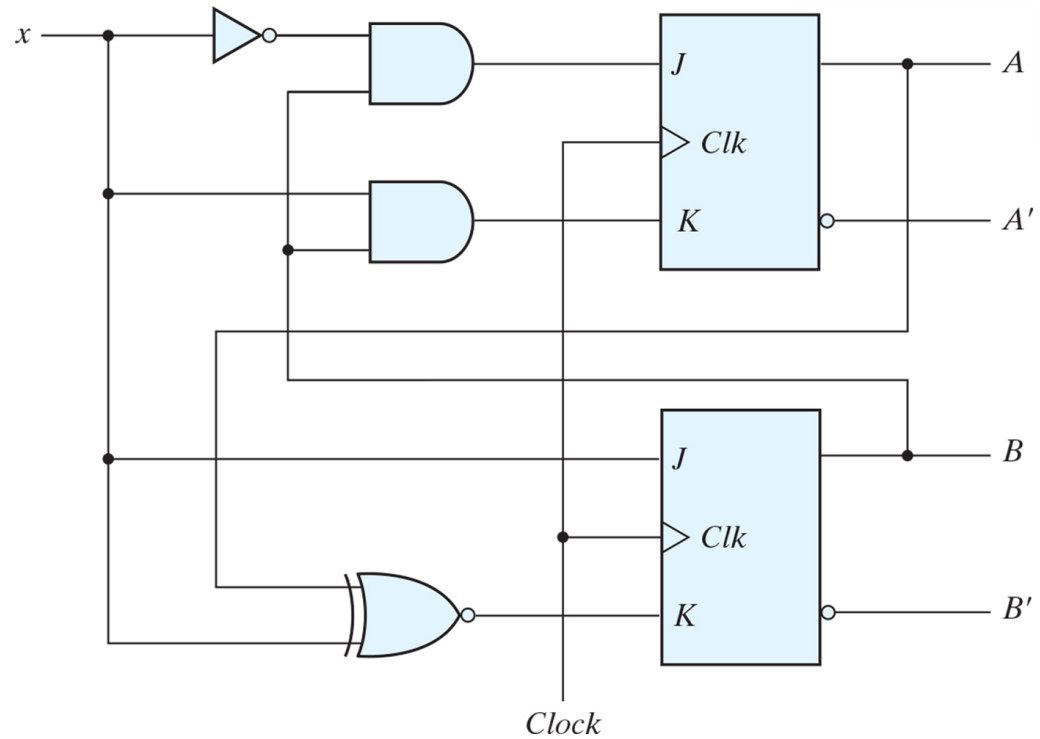
$K_B = (A \oplus x)'$

$$J_A = Bx'$$

$$K_A = Bx$$

$$J_B = x$$

$$K_B = (A \oplus x)'$$



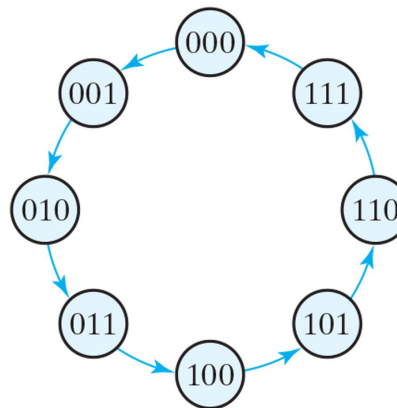
❖ 설계 예제 - T 플립플롭

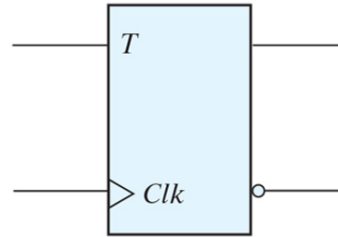
입력 : Clk Only

출력 : 상태(3-bit)

기능 : Up Counter(0~7), 7 표기 후 0으로 순환

플립플롭 : T 플립플롭 사용

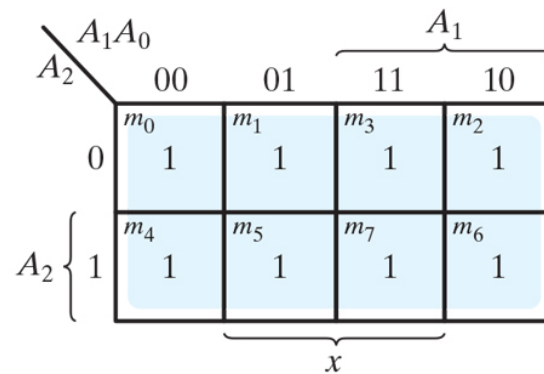
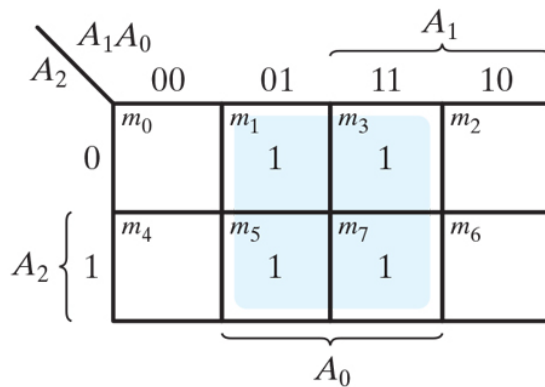
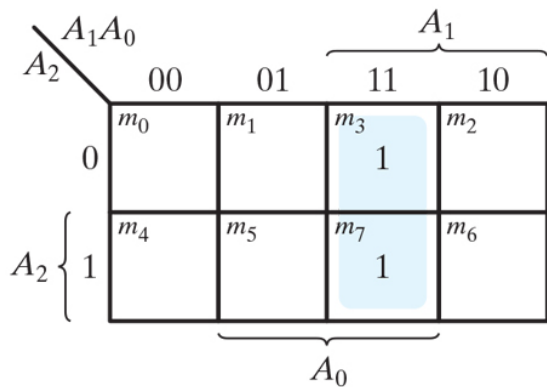


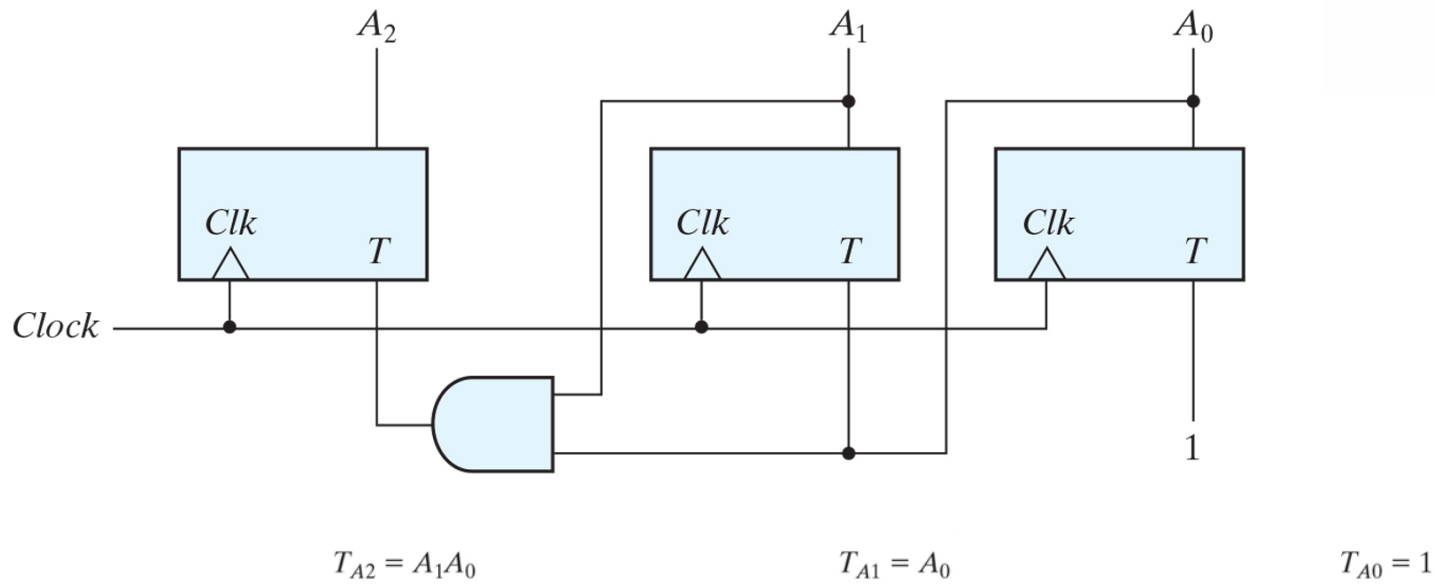


Present State			Next State			Flip-Flop Inputs		
A_2	A_1	A_0	A_2	A_1	A_0	T_{A2}	T_{A1}	T_{A0}
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	1
0	1	0	0	1	1	0	0	1
0	1	1	1	0	0	1	1	1
1	0	0	1	0	1	0	0	1
1	0	1	1	1	0	0	1	1
1	1	0	1	1	1	0	0	1
1	1	1	0	0	0	1	1	1

$Q(t)$	$Q(t = 1)$	T
0	0	0
0	1	1
1	0	1
1	1	0

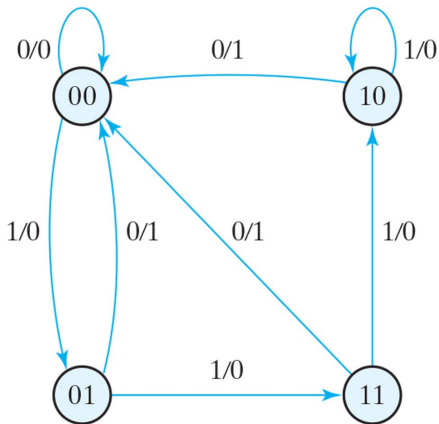
Present State			Next State			Flip-Flop Inputs		
A_2	A_1	A_0	A_2	A_1	A_0	T_{A2}	T_{A1}	T_{A0}
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	1
0	1	0	0	1	1	0	0	1
0	1	1	1	0	0	1	1	1
1	0	0	1	0	1	0	0	1
1	0	1	1	1	0	0	1	1
1	1	0	1	1	1	0	0	1
1	1	1	0	0	0	1	1	1





❖ 연습문제 5.11

- (a) 입력 시퀀스 110010100111010 이 회로에 인가, 초기상태(00), 상태전이 및 출력 시퀀스 결정
 (b) 상태표 도출 후 상태 축소
 (c) 축소된 상태표가 같은 지 검증



A = 00 B = 01
 C = 10 D = 11

상태	00															
입력	1	1	0	0	1	0	1	0	0	1	1	1	0	1	0	
출력																

현재	다음		출력	
	X=0	X=1	X=0	X=1
A				
B				
C				
D				

❖ 연습문제 5.11

(b) 상태표 도출 후 상태 축소

현재	다음		출력	
	X=0	X=1	X=0	X=1
A	A	B	0	0
B	A	D	1	0
C	A	C	1	0
D	A	C	1	0

현재	다음		출력	
	X=0	X=1	X=0	X=1
A	A	B	0	0
B	A	B	1	0

❖ 연습문제 5.11

(c) 축소 된 상태표가 같은 지 검증

상태	A 00	B 01	D 11	A 00	A 00	B 01	A 00	B 01	A 00	A 00	B 01	D 11	C 10	A 00	B 01
입력	1	1	0	0	1	0	1	0	0	1	1	1	0	1	0
출력	0	0	1	0	0	1	0	1	0	0	0	0	1	0	1

$$D = C = B$$

상태	A 00	B 01	B 01	A 00	A 00	B 01	A 00	B 01	A 00	A 00	B 01	B 01	B 01	A 00	B 01
입력	1	1	0	0	1	0	1	0	0	1	1	1	0	1	0
출력	0	0	1	0	0	1	0	1	0	0	0	0	1	0	1

현재	다음		출력	
	X=0	X=1	X=0	X=1
A	A	B	0	0
B	A	B	1	0

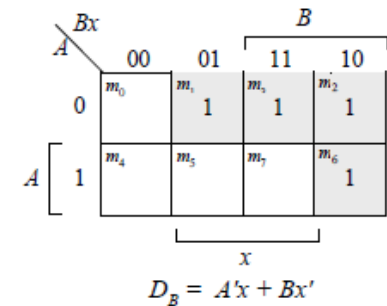
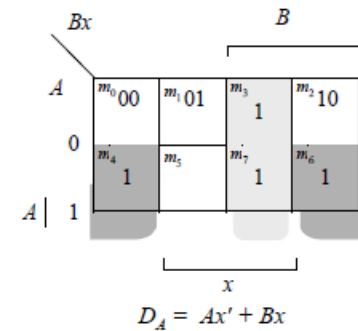
❖ 연습문제 5.16

2개의 D플립플롭 A, B 그리고 입력 x를 가진 순차회로 설계 (a, b 동일한 유형의 문제로 a만 풀이)

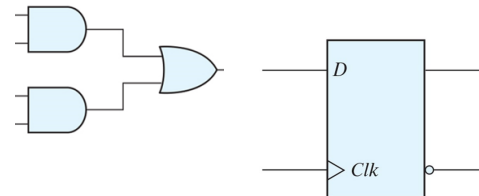
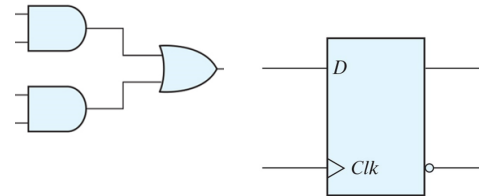
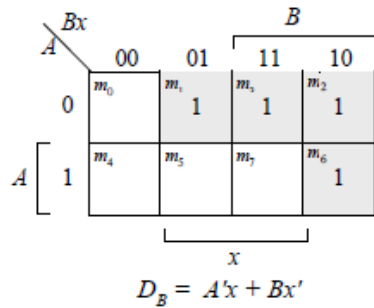
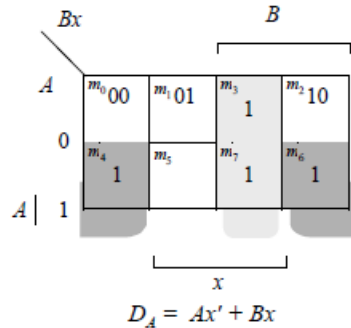
(a) $x=0$ 일 때 현재상태 유지, $x=1$ 일 때 00 - 01 - 11 - 10 - 00 순환

(b) $x=0$ 일 때 현재상태 유지, $x=1$ 일 때 00 - 11 - 01 - 10 - 00 순환

A	B	x	A	B
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	1
1	0	0	1	0
1	0	1	0	0
1	1	0	1	1
1	1	1	1	0



❖ 연습문제 5.16



❖ 과제

연습문제 5.6, 5.7, 5.12, 5.17

다음 강의주차에 풀이 제공 예정

